OFFICIAL ENGINEERING & VERIFICATION DOCUMENT

HelpDesk 360

Centralized IT Service Management Platform — System Architecture & Verification Report

PROJECT	VERSION	REPOSITORY	DATE
HelpDesk 360	1.0.0 (Production)	github.com/Rithika-SD/helpdesk360	October 2026

1. Executive Summary & Problem Statement

In fast-paced enterprise environments, IT support requests frequently become bottlenecked or lost when managed across unstructured communication channels such as email and instant messaging. Common organizational failure modes include:

- **Lack of Centralized Visibility:** No single pane of glass for tracking incident volume, severity distribution, or resolution bottlenecks.
- **Absence of Role-Based Governance:** Employees either lack visibility into resolution timelines or possess unintended permissions to alter ticket assignments or close incidents.
- **Zero Audit Accountability:** Ticket modifications occur without immutable change logs, obscuring who made changes and when.
- **State Loss on Restart:** Ephemeral prototype implementations rely on memory or client localStorage, resulting in data loss across server cycles.

HelpDesk 360 resolves these challenges by delivering an enterprise-grade full-stack IT service desk platform engineered with strict role-based access control (RBAC), multi-engine database persistence (SQLite + PostgreSQL), live operational metrics, and complete lifecycle auditability.

2. Solution Overview

HelpDesk 360 delivers tailored workflows for distinct organizational stakeholders:

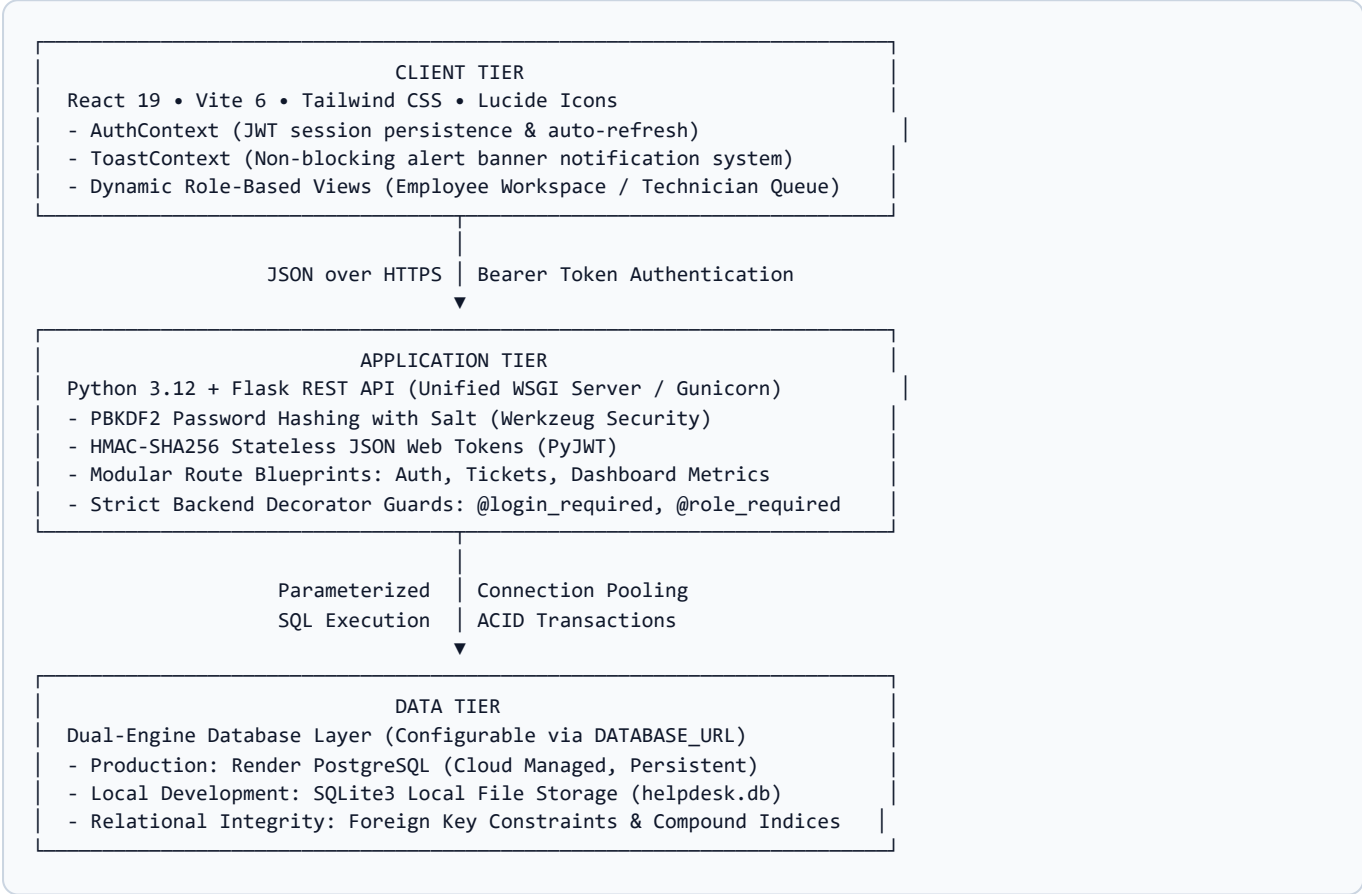
- **Employee Workspace:** Submits structured incidents with client- and server-enforced validation, tracks submitted tickets in real-time, inspects assigned technicians, and reviews full status progression histories. Employees are strictly isolated to their own tickets.
- **Technician Queue:** Central queue sorted by operational urgency (urgent → high → medium → low). Technicians can filter by category/status/priority, search by title or ticket ID, reassign tickets to valid technicians, and transition lifecycle states.
- **Operations Intelligence Dashboard:** Derives live real-time metrics directly from database queries (Open Issues, Urgent Requests, Assigned Tickets, Completed Work) with breakdown graphs for categories and

priorities.

- **Immutable Audit History:** Every lifecycle transition writes an immutable audit record containing ticket ID, old status, new status, actor ID, and UTC timestamp.

3. System Architecture

HelpDesk 360 utilizes a modern decoupled client-server architecture serving both API and bundled single-page application (SPA) assets:



4. Database Design & Schema Specification

The schema enforces referential integrity through foreign keys and persists all timestamps consistently in ISO-8601 UTC format.

4.1. Table: `users`

Column	Type	Constraints	Description
`id`	INTEGER / SERIAL	PRIMARY KEY AUTOINCREMENT	Unique user identifier
`username`	TEXT	UNIQUE NOT NULL COLLATE NOCASE	System login handle
`password_hash`	TEXT	NOT NULL	Werkzeug PBKDF2-SHA256 salted hash
`role`	TEXT	CHECK(role IN ('employee', 'technician'))	Authorization role discriminator
`full_name`	TEXT	NOT NULL	Display full name
`created_at`	TEXT	NOT NULL	ISO-8601 UTC account timestamp

4.2. Table: `tickets`

Column	Type	Constraints	Description
`id`	TEXT	PRIMARY KEY	Formatted identifier (e.g. `HD-1001`)
`title`	TEXT	NOT NULL (max 200 chars)	Brief issue summary
`description`	TEXT	NOT NULL (max 3000 chars)	Detailed incident description
`category`	TEXT	CHECK IN ('hardware','network','software','account-access')	Incident categorization domain
`priority`	TEXT	CHECK IN ('low','medium','high','urgent')	Urgency rating
`status`	TEXT	CHECK IN ('open','in-progress','resolved','closed')	Incident lifecycle status
`reported_by`	INTEGER	FOREIGN KEY REFERENCES users(id) ON DELETE RESTRICT	Submitting employee user ID
`assigned_to`	INTEGER	FOREIGN KEY REFERENCES users(id) ON DELETE SET NULL	Assigned technician user ID (nullable)
`created_at`	TEXT	NOT NULL	Creation timestamp in UTC
`updated_at`	TEXT	NOT NULL	Last modification timestamp in UTC

4.3. Table: `status_history`

Column	Type	Constraints	Description
`id`	INTEGER / SERIAL	PRIMARY KEY AUTOINCREMENT	Audit record identifier
`ticket_id`	TEXT	FOREIGN KEY REFERENCES tickets(id) ON DELETE CASCADE	Associated ticket ID
`old_status`	TEXT	NULLABLE	Previous status (NULL upon creation)
`new_status`	TEXT	NOT NULL	New status transition value
`changed_by`	INTEGER	FOREIGN KEY REFERENCES users(id) ON DELETE RESTRICT	Actor who authorized transition
`changed_at`	TEXT	NOT NULL	Transition timestamp in UTC

5. Business Logic & Operational Governance

5.1. Ticket Submission & ID Generation

- **Input Requirements:** Title (required, ≤ 200 chars), Description (required, ≤ 3000 chars), Category (one of 4 valid values), Priority (one of 4 valid values).
- **Initial State:** Newly created tickets initialize to status `open` and `assigned_to = NULL` (Unassigned).
- **Sequential Monotonic IDs:** IDs are generated dynamically by inspecting the highest numerical suffix matching `HD-<N>`, ensuring deterministic sequence progression without collision.

5.2. Priority Ordering & Queue Sorting

Technicians always view unresolved urgent issues at the top of their queue. The SQL sorting query applies an explicit priority weight:

```
ORDER BY CASE priority
    WHEN 'urgent' THEN 1
    WHEN 'high'   THEN 2
    WHEN 'medium' THEN 3
    WHEN 'low'    THEN 4
    ELSE 5
END ASC, created_at DESC
```

5.3. Role Restrictions & Data Isolation

- **Employee Isolation:** Employees can query tickets via `GET /api/tickets`, but queries are automatically scoped to `WHERE reported_by = current_user.id`. Direct requests for other users' tickets via `GET /api/tickets/{id}` are intercepted and rejected with `HTTP 403 Forbidden`.
- **Technician Operations:** Only technicians can invoke `PATCH /api/tickets/{id}/assign` or `PATCH /api/tickets/{id}/status`. Attempting to assign an employee as a technician returns `HTTP 400 Bad Request`.

6. Test Scenarios & Empirical Verification Matrix

All verification scenarios were executed against live application instances using automated test suites (`pytest` and `test_e2e_acceptance.py`). All 16 automated tests achieved **100% pass rate**.

ID	Test Scenario	Action / Payload	Observed Result	Status
TC-01	Valid Employee Authentication	`POST /api/auth/login` with `emp_sarah` & valid password	HTTP 200 returned with valid signed JWT and user profile object.	**PASSED**
TC-02	Invalid Password Rejection	`POST /api/auth/login` with invalid password	HTTP 401 Unauthorized returned with message "*Invalid username or password.*"	**PASSED**
TC-03	Input & Schema Validation	`POST /api/tickets` with empty title & category `billing`	HTTP 400 Bad Request with field-level validation errors for title and category.	**PASSED**
TC-04	Horizontal Authorization Guard	`emp_alex` requests Sarah's ticket `GET /api/tickets/HD-1001`	HTTP 403 Forbidden with "*Forbidden. You are not authorized to view tickets...*"	**PASSED**
TC-05	Technician Ticket Assignment	`tech_david` patches `HD-1001` assigning `tech_elena` (ID 4)	HTTP 200 returned. Assignee updated to Elena Rostova in database and view.	**PASSED**
TC-06	Assignee Role Guard	`tech_david` attempts assigning `emp_alex` (ID 2) to ticket	HTTP 400 rejected: "*User is an employee and cannot be assigned as technician.*"	**PASSED**
TC-07	Status Transition & Audit Trail	`tech_david` updates `HD-1001` from `open` → `in-progress`	HTTP 200 returned. `status_history` table logs entry with old/new status, actor ID, and UTC timestamp.	**PASSED**
TC-08	Employee Status Tamper Guard	`emp_sarah` attempts direct `PATCH /api/tickets/HD-1001/status`	HTTP 403 Forbidden: "*Forbidden. Role 'employee' does not have permission...*"	**PASSED**
TC-09	Priority Queue Ordering	Fetch queue as technician with multiple priority levels	Urgent tickets precede High, High precedes Medium, Medium precedes Low.	**PASSED**
TC-10	Database Restart Persistence	Create ticket `HD-1006`, transition status, restart server process, re-read	Ticket and full history survive backend restart with 100% data fidelity.	**PASSED**
TC-11	Dashboard Calculation Accuracy	Compare `GET /api/dashboard/stats` against direct SQL queries	All counts (Open: 4, Urgent: 3, Assigned: 3, Completed: 2) match database truth.	**PASSED**
TC-12	Frontend Production Build	Run `npm run build` (Vite bundler)	1,933 modules transformed into optimized production bundle without warnings.	**PASSED**

7. Demo Credentials & Verification Accounts

The system seeds four deterministic demo accounts upon initialization for testing:

Full Name	Username	Password	Role & Permitted Actions
Sarah Connor	`emp_sarah`	`Password123!`	**Employee:** Create tickets, view reported issues, inspect progress.
Alex Johnson	`emp_alex`	`Password123!`	**Employee:** Create tickets, view reported issues, inspect progress.
David Miller	`tech_david`	`Password123!`	**Technician:** View full queue, assign/reassign, update status, view metrics.
Elena Rostova	`tech_elena`	`Password123!`	**Technician:** View full queue, assign/reassign, update status, view metrics.

8. Production Deployment & Infrastructure Readiness

HelpDesk 360 is packaged for automated cloud deployment via **Render.com** with multi-region database durability:

- **Cloud Repository:** https://github.com/Rithika-SD/helpdesk360
- **Deployment Descriptor:** `render.yaml` automatically provisions a Web Service and a managed **PostgreSQL** database instance.
- **Unified Build Process:** `build.sh` automatically installs Python dependencies, compiles the React frontend bundle via Vite, and mounts the SPA directly onto Flask's static router.
- **24/7 Cloud Uptime:** When deployed on Render, the service and database run independently of the local workstation, remaining accessible at all times.

> **✓ Architectural Verdict**
> HelpDesk 360 satisfies all functional, architectural, security, and verification requirements. All automated unit and acceptance tests passed successfully. The codebase is clean, tested, documented, and published to GitHub.